The Performance Commons

Why ASIC-level performance is a collective-action problem, and the hardware we should build for its solution

A position paper. The underlying architecture analysis is drawn from the companion technical comparison, "Dataflow vs. an Optimal Heterogeneous Architecture."

Abstract

The reason we don't achieve near ASIC performance and power (and the reason we don't have much better ASICs) for many more applications is not technical in nature. It is the collective action problem both within and across levels of the computing stack. As machine intelligence continues to advance, as long as it maintains the apparent inoculation to this problem, this fact will increasingly be exposed. This raises many questions for forward looking engineers in the form of, "How do we design systems assuming the collective action problem is solved?". For example, how do we design hardware assuming there will be orders of magnitude more intelligence available for shared code optimization? We likely want more reconfigurable and less "programmable" architectures. And similarly, how do we design software stacks assuming the programmability issues holding back wider adoption of distributed computing are solved?

1. The gap is real, and it is not physics

A general-purpose core is roughly **100–500× less energy-efficient than an ASIC** for the same task (Hameed et al., ISCA 2010). Word-level reconfigurable hardware (a CGRA) sits within ~2.8× of an ASIC; bit-level reconfigurable hardware (an FPGA) ~20–35× off (Prabhakar et al., ISCA 2017; Kuon & Rose, 2007). Most general-purpose code runs one to two orders of magnitude below the achievable frontier.

The companion analysis shows this gap is not bound by physics. A perfect compiler and scheduler brings each substrate to its roofline, and the classic limit studies (Lam & Wilson, ISCA 1992; Wall, ASPLOS 1991) locate the dominant barrier in *control flow and the effort of exposing parallelism* — not in raw data dependence. The headroom is real and substrate-reachable. What separates achieved from achievable is therefore, overwhelmingly, the **effort of specialization** — writing the optimal kernel, building the accelerator, tuning to the platform — and not a wall in the silicon.

Substrate	Penalty vs an ASIC (same function)	Measured by
ASIC	1× — the efficiency ceiling	—
CGRA (word-level reconfigurable)	~2.8× area	Prabhakar et al. (Plasticine), ISCA 2017
FPGA (bit-level reconfigurable)	~20–35× area, ~3–4× delay, ~10–14× power	Kuon & Rose, 2007
General-purpose CPU	~100–500× energy	Hameed et al., ISCA 2010

Table 1. The measured cost of generality. These are the intrinsic, physical penalties — the realizable frontier of Figure 1 — not what most code actually achieves, which is typically worse because the substrate's potential goes unextracted.

2. The general-purpose vs. custom trade-off is mostly an effort artifact

The trade-off at the center of computer architecture is usually drawn as a Pareto frontier between **flexibility** and **efficiency**: a CPU does anything but wastes energy; an ASIC wastes nothing but does one thing; FPGAs and CGRAs sit in between. The conventional reading is that you *buy* efficiency with flexibility and pick your point on the curve. But that curve is drawn at a fixed, finite *programmer effort* — it is a slice through a three-dimensional surface (flexibility × efficiency × effort), and the slice moves as effort changes.

To see why, decompose the efficiency gap between a reconfigurable substrate and an ASIC into two parts. The first is the **intrinsic fabric tax**: the physical overhead of the reconfigurable silicon itself — the switches, the configuration memory, the over-general datapath. This is irreducible; no amount of programmer effort removes it, and it is what the measured ratios capture (a CGRA ~2.8× an ASIC's area; an FPGA ~20–35×). The second is the **realization gap**: how much of the substrate's *available* efficiency you actually extract, which depends entirely on the quality of mapping, placement, and scheduling — and therefore on effort. At finite effort you extract a fraction; at the limit (the oracle mapping of the companion report) you extract all of it.

The consequence is the whole argument in one picture. As effort rises, the realization gap closes and the achieved-efficiency curve falls toward the realizable frontier — which is just the intrinsic fabric tax. Crucially, **the effort gap is concentrated exactly on the reconfigurable substrates**: it is ~zero at the custom end (an ASIC has nothing to program) and small at the general-purpose end (CPUs are easy to target and their compilers are mature), but it is enormous in the middle, where reconfigurable hardware lives. This is precisely why reconfigurable fabrics have historically underdelivered — not because they are inefficient, but because their efficiency was unreachable at affordable effort. Close that gap — which is what a shared, AI-driven optimization commons does — and a CGRA sits within ~2.8× of an ASIC *while keeping full flexibility.* The flexibility–efficiency trade-off does not so much shift as **flatten**: under unlimited effort, the only efficiency you still surrender for flexibility is the small, physical fabric tax.

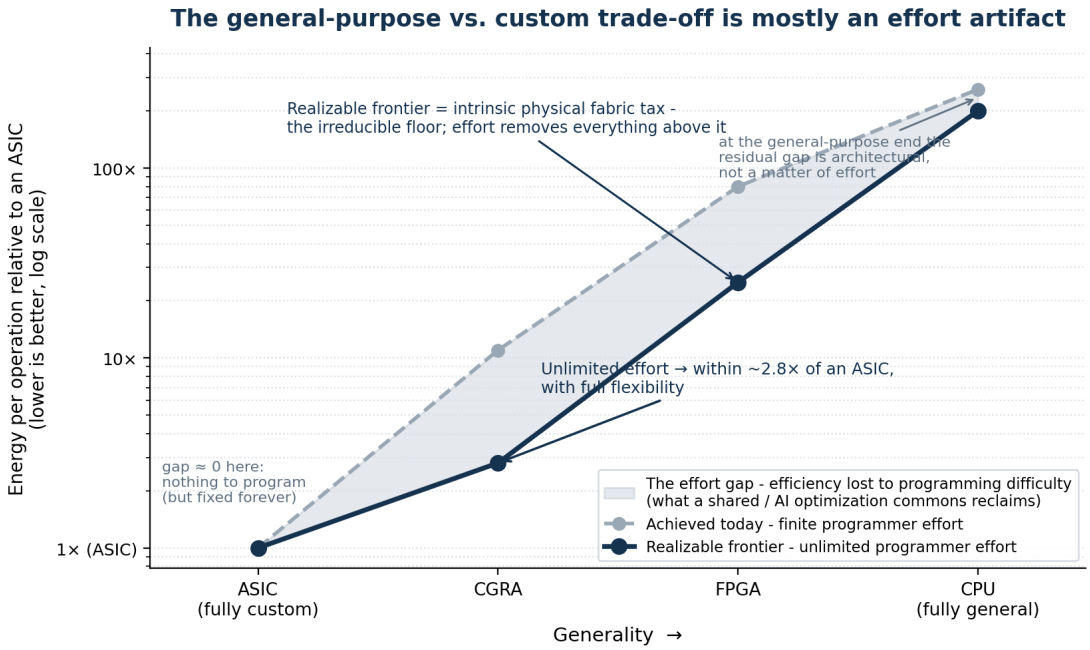

Figure 1. The canonical flexibility–efficiency trade-off is largely an artifact of finite programmer effort. The shaded band is the effort gap — efficiency forfeited because extracting a reconfigurable substrate's potential is too laborious to afford per workload. It is widest exactly where reconfigurable hardware sits, ~zero at the custom end (nothing to program), and architectural rather than effort-bound at the general-purpose end. A shared / AI optimization commons reclaims the band; what remains is the intrinsic physical fabric tax (CGRA ~2.8×, FPGA ~20–35×). Substrate-to-ASIC ratios on the realizable frontier are anchored to measured values; the finite-effort curve and the effort axis are illustrative.

One more turn of the screw closes the loop with the design conclusion. Even *unlimited design effort* — machine intelligence spinning up bespoke ASICs cheaply — does not argue for ASICs everywhere, because an ASIC is fixed at tape-out and real workloads are not: the adaptability an ASIC structurally lacks is exactly what its efficiency edge over a CGRA costs. So unlimited effort, applied honestly, does not push the optimum toward *more custom* silicon; it pushes it toward **reconfigurable** silicon that an oracle-grade commons can drive to within a small factor of custom while remaining able to change what it computes. That is the analytical core of "more reconfigurable, less programmable."

3. Distributed computing is the same problem, one level up

The abstract's second question — how to design software stacks assuming distributed computing's programmability barriers are solved — is the same collective-action problem displaced from the chip to the network, and the latent resource is enormous and measurably wasted. Even inside professionally run datacenters, average server utilization sits at **12–18%**, active servers rarely exceed 50%, and up to **30% of servers are "comatose"** — powered, cooled, and depreciating while doing no useful work (NRDC / Anthesis); one recent estimate puts ~10 million fully idle servers at roughly $30B in stranded capital. Outside the datacenter the waste is larger still: billions of consumer devices sit idle most of the time, having already paid their hardware, power, and bandwidth costs. The compute exists. What is missing is any mechanism to pool it.

It goes unpooled for exactly the reasons §2's effort gap goes unpaid: coordinating loosely-coupled, heterogeneous, intermittent hardware is hard to program, and no individual is incentivized to contribute idle cycles to a shared pool. Naive methods that communicate every step waste almost all of a loose federation's compute; methods built for loose coupling reclaim it. DiLoCo (Google DeepMind, 2023) trains language models across poorly connected islands while communicating ~500× less than synchronous training; OpenDiLoCo replicated this across two continents and three countries at 90–95% utilization, with nodes joining and leaving mid-run; DiLoCoX scaled the approach to a 107-billion-parameter model; and Hivemind and Petals run training and inference of very large models over thousands of volunteer nodes. The barrier was never the hardware (Figure 2).

What makes solving it worth the effort is the demand side: cheap, unreliable compute is only valuable if something will consume it, and machine intelligence consumes without bound. Human compute demand is finite — businesses have finite workloads, people have finite needs, and in equilibrium price competition retires high-cost suppliers. Machine demand is not finite, because there is always a next-best task: a machine that cannot profitably run a $50/hr job can still create value on a $5/hr job, and below that on speculative search, precomputation, and self-improvement, down to arbitrarily low value. Humans run out of ideas; machines run out of compute. This turns the market permanently *undersupplied*, which is precisely the condition under which cheap, unreliable home compute — roughly **6× cheaper than a datacenter** (~$0.08 vs ~$0.50 per hour, since the hardware and power are already sunk) — creates genuine value instead of being undercut (Figure 3).

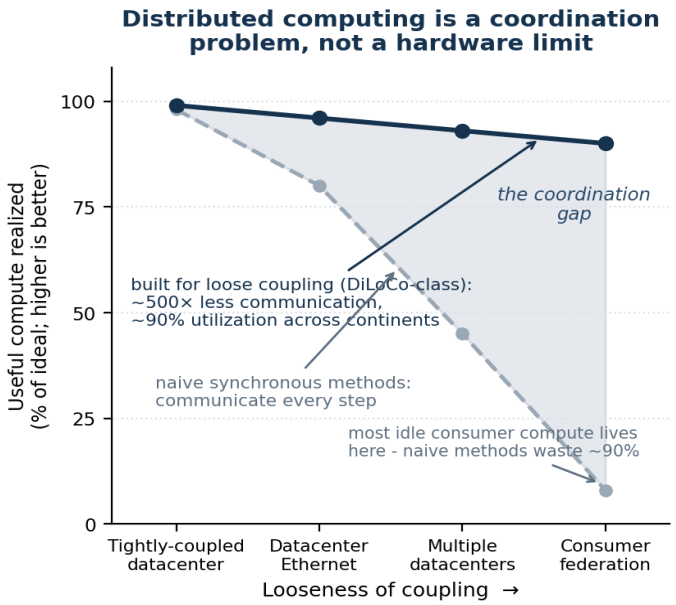

Figure 2. The same shape of problem, one level up. The aggregate idle compute in consumer devices is latent not for hardware reasons but because coordinating loosely-coupled, heterogeneous, intermittent hardware is hard to program — and because no individual is incentivized to pool it. Naive (communicate-every-step) methods waste almost all of it at the loosely-coupled end; methods built for loose coupling (DiLoCo and successors: ~500× less communication, ~90% utilization across continents) reclaim it. As in Figure 1, the gap is concentrated where the resource is hardest to reach, and it is an artifact of programmability and coordination, not physics. Utilization figures are illustrative, anchored to reported DiLoCo / OpenDiLoCo results.

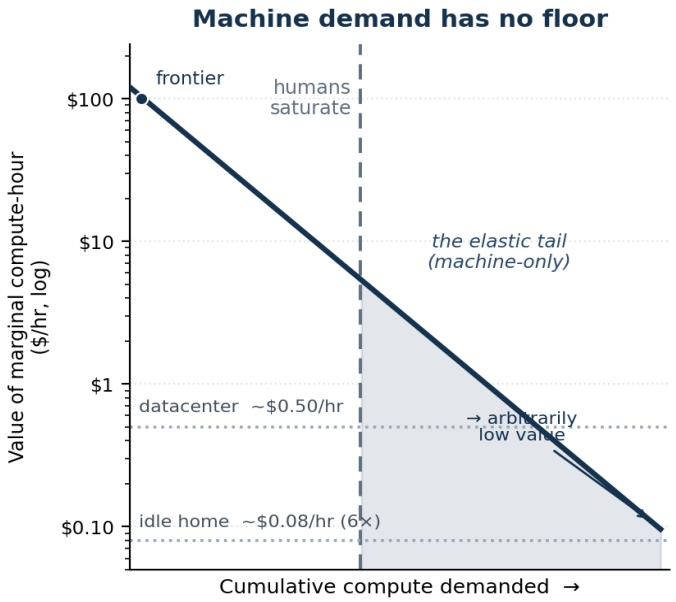

Figure 3. Why the idle resource becomes valuable. Human demand for compute saturates once the worthwhile tasks are done; machine demand does not, because there is always a lower-value next-best task, so the demand curve extends toward arbitrarily low value. That tail falls below datacenter economics (~$0.50/hr) into the range only cheap idle compute (~$0.08/hr, ~6× cheaper) can serve — which is what makes a federation of otherwise-wasted devices worth assembling. The curve is illustrative; the cost anchors come from Omerta's reliability-market simulation and the task-value ladder from its economic analysis.

This is not only an argument; the author's companion project **Omerta** is an attempt at the software side. Its networking substrate — a peer-to-peer mesh providing end-to-end encryption, NAT traversal and relaying, gossip-based peer discovery, and tunnelling — is the most built; ephemeral-VM isolation and a discrete-event economic simulator work; and the piece that ties everything together, an end-to-end consumer-to-provider compute session over the mesh, is mid-migration and not yet working end to end. (Its current state, set beside the other efforts this paper draws on, is laid out in §12.) The wager is this paper's: Golem, iExec, and BOINC stalled on human friction, consensus overhead, and extractive token economics, and machine intelligence is what removes those barriers. Omerta is thus neither vapor nor a finished platform — which is the point: the abstract's software-stack question is concrete enough to be under construction, its hardest parts coordination and integration, not feasibility.

4. Near-oracular performance is a privilege

Consider who actually reaches the frontier today: hyperscalers with custom silicon, high-frequency-trading firms, AAA game-engine teams, codec and DSP vendors, and the handful of groups who hand-write CUDA or assembly. They get there through enormous, bespoke, and largely unshared effort, justified only by scale or stakes. Everyone else ships code far off the frontier — not for lack of talent or physics, but because the cost of specialization is affordable only at those scales. Performance is gated by *who can pay the specialization cost*, not by what is achievable.

5. The privilege is a collective-action problem

An optimization is a **non-rival good**: my hand-tuned FFT does not diminish yours. Classic economics says non-rival goods are under-provided whenever no single actor captures enough of the aggregate benefit to justify producing them for the whole population. So each team re-derives or hoards, and the population sits below the frontier. This is the textbook structure of a collective-action problem — under-provision of a public good. The performance gap is not a technology gap; it is a coordination failure.

6. The same logic explains the absence of better ASICs

One level down, ASIC *design and optimization* knowledge — mapping heuristics, verified IP blocks, the place-and-route and microarchitectural tricks that separate a good datapath from a mediocre one — is equally non-rival and equally hoarded. EDA vendors are a partial, proprietary aggregation point; the rest is re-derived bespoke per chip and locked inside firms. So the field produces fewer and worse ASICs than it collectively could, for the same reason: the design effort is not pooled.

This is the aggressive claim, so state the caveat plainly: fab economics and the genuine bespokeness of different computations are also real costs, and pooling cannot erase them. The claim is *not* that collective action is the only barrier — it is that the **pooling-addressable fraction** of the gap is large and currently wasted.

Layer of the stack	The hoarded / secret non-rival good	Open-commons counter-trend
Foundry / process	PDKs & device models under NDA	SkyWater & IHP open 130 nm PDKs
CAD / EDA tooling	Synthesis, place-&-route, timing heuristics and their new RL optimizers (Synopsys/Cadence/Siemens; DSO.ai, Cerebrus)	OpenROAD, OpenLane, Yosys
Hardware IP blocks	Verified IP licensed proprietary	OpenCores, CHIPS Alliance, OpenHW CORE-V; Rocket/BOOM/CVA6
ISA	Proprietary ISAs (x86, ARM)	RISC-V
Microcode / firmware	Encrypted microcode, proprietary blobs	coreboot/LinuxBoot, OpenBMC, OpenTitan
Math / ML kernels	Fastest per-chip kernels hoarded (MKL, cuBLAS, cuDNN)	OpenBLAS, BLIS; TVM/Halide/Triton/Exo
GPU drivers / runtime	Per-application driver optimizations as a moat	Mesa, NVK
Workload / measurement data	Real traces siloed in the firms that own the fleets	Google cluster traces, MLPerf
Distributed compute	Idle compute unharnessed — hard to program *and* no incentive to pool	BOINC/Folding@home; DiLoCo/Hivemind/Petals

Table 2. One pattern, every layer. At each level a non-rival good — design knowledge, optimization, measurement — is hoarded or secret, and at each level an open commons is forming to route around it. The top five rows are really a single problem: the hardware-design commons that does not yet exist, from secrecy in the foundry's PDKs up through CAD tooling and IP to the ISA itself.

7. Open source already solved this — and did it by itself

The compiler and library layer is a *solved instance* of exactly this problem. LLVM, GCC, the Linux kernel, and BLAS/LAPACK are pooled optimization commons: every program compiled with LLVM inherits the accumulated optimization effort of thousands of contributors it never paid for and could never have produced alone. Crucially, this commons formed **organically**, because the cost of contributing to a shared optimizer is far below the benefit everyone draws once it exists. The trajectory is established. The only open question is how far up the stack it extends — from instruction selection (already done) into algorithm selection, runtime specialization, and ultimately hardware mapping.

Open commons	Out-competed	Fitness evidence	Layer
Linux	Proprietary UNIXes (Solaris, AIX, IRIX)	100% of the TOP500; dominates cloud & servers; Android	OS
LLVM	Proprietary / fragmented compiler back-ends	Substrate for Clang, Swift, Rust, Julia; industry-wide	Compiler
RISC-V	Proprietary ISAs (x86, ARM)	13B+ cores by end-2023; Qualcomm 650M+, NVIDIA's CUDA cores on RISC-V; Esperanto's 1,088-core chip taped out without ~$25–30M in ARM fees	ISA
Mesa	Proprietary GPU drivers	Open driver stack reaching/beating proprietary parity on Linux	Driver
PyTorch / TensorFlow	Proprietary ML toolkits	The substrate of essentially all ML research and much production	ML framework
FFmpeg	Proprietary media / codec libraries	Underlies most of the world's media processing	Codec

Table 3. The commons wins at six different layers. Each out-competed proprietary, siloed alternatives — the pattern the paper argues is now climbing toward hardware.

The reason is structural: open commons have higher fitness in the literal variation-selection-inheritance sense. More contributors generate more variation than any single firm's team; forking is selection without permission, so good branches are adopted and bad ones die with no proprietary roadmap gating exploration; zero marginal cost of replication plus network effects spread the fitter variant and accrete an ecosystem that raises switching costs; and there is no single point of strategic death — a company can be acquired, pivot, or die and take its tooling with it, whereas a commons persists and forks, accumulating improvement over many more generations. Each improvement is inherited by every future user. That is why, once viable, the commons tends to win — and it is why expecting it to keep climbing the stack is not wishful but the default. The one qualifier sharpens the prediction: the commons wins where the artifact is non-rival and modular, contribution cost is low relative to the shared benefit, and no capital or data moat dominates — which is exactly why it has already taken compilers, the OS, and the ISA, is taking drivers and kernels now, and has not yet cracked leading-edge EDA or fabs, where capital and tacit secrecy are the moat.

That last moat is where machine intelligence changes the arithmetic first. A model can be pointed at an open flow — read it, profile it, rewrite its heuristics — in a way it can never be pointed at a licensed binary. And this is no longer hypothetical: LLM-driven improvement of optimization code is established practice — Evolution through Large Models (Lehman et al., 2022), FunSearch (DeepMind, Nature 2024), AlphaEvolve (DeepMind, 2025; in production, a single recovered scheduling heuristic reclaims ~0.7% of Google's fleet compute) — and it has reached open EDA specifically: LLM agents tuning the OpenROAD flow now beat Bayesian optimization (ORFS-agent, Ghose et al., 2025), and coding agents are being turned on OpenROAD's own code (2026 preprint). The proprietary vendors ship the same capability inside the license — Synopsys DSO.ai (2020), Cadence Cerebrus (2021) — proving the value while

hoarding it, one more row of Table 2. Machine intelligence collapses exactly the contribution cost that kept EDA knowledge tacit and locked inside firms; only in an open flow does the result compound for everyone.

8. Machine intelligence multiplies the commons; it does not replace it

It is tempting to think that sufficiently capable AI makes the commons unnecessary — each team's AI simply specializes its own code. This is the wrong conclusion, for a specific reason: in the near term, reaching near-oracular performance still takes significant effort *even with machine intelligence in the loop*. AI lowers the per-instance cost of specialization; it does not drive it to zero.

So AI does not dissolve the collective-action problem — it raises the stakes of solving it. The thing being pooled becomes more powerful per unit (AI-generated optimizations alongside human ones), which makes the commons *more* valuable, not less. **AI plus pooling compounds; AI alone, hoarded per team, simply makes a few elite groups faster and widens the gap.** Capability is not the lever. Collaboration is. This is the sense in which machine intelligence is *inoculated* against the collective-action problem: a shared or copyable model aggregates optimization knowledge across the whole population without the incentive to hoard that defeats human coordination — and as it does, it exposes how much of the performance gap was only ever a coordination failure.

There is early measured signal that the multiplier is real. Measured from raw git history, one person directing a fleet of agents reaches team scale — on the order of 250–1,250 Python-equivalent lines per working hour against a human hand-coding norm of 10–50, matching open-source projects carrying several hundred contributors day for day. The largest such burst was built with an *off-the-shelf* assistant, not bespoke tooling — evidence the capability rides in the *general, shared* model available to everyone, which is precisely the population-wide aggregation just described. The honest asterisk is the one the measurement itself foregrounds: human review is the one input that does not scale with model speed, the defect rate rises as output outpaces review, and rigorous studies of the *one-developer-plus-assistant* case have found null or negative effects — the multiplier appears under fleet orchestration, not solo assist, and the burden of proof remains on it.

Which way the multiplier cuts is itself an open question. A model everyone routes their ideation through is a *consensus-amplifier* — it raises apparent coherence while cutting real diversity, and the homogenization compounds as adoption spreads; the same technology is a *coherence subsidy* only if it makes coordination cheap enough that a population can hold the same agreement at *higher* idea-diversity. That fork is the precise content of *multiplies the commons, does not replace it*: the machine aggregates the population's variation but does not originate it, so collaboration, not capability, decides whether the commons widens or collapses — a tension §11 returns to.

9. The aggregation point already exists

A collective-action problem is solved by an actor with the **vantage**, the **channel**, and the **incentive** to internalize the externality. The hardware-and-runtime vendor layer has all three: it is the common substrate (it alone sees the whole workload distribution — the only vantage from which population-level optimization is even possible); it owns the distribution channel into the optimization layer (microcode, drivers, firmware, on-device runtimes); and it captures the aggregate benefit (real-world performance on its silicon converts directly to competitive advantage).

This is already happening in fragments. GPU vendors ship per-application optimization profiles, aggregated across all users of a title, through driver updates. Mobile runtimes ship cloud profiles that pre-optimize a fresh install from the population's hot paths. The vertically integrated vendor that owns silicon, OS, compiler, and frameworks is the existence proof of the full stack. The "processor that reads code as a contract" is the generalization: hardware that honors the *user-visible outcome* and is free to choose any legal behavior satisfying it, with the optimization policy improving across the federation.

10. The implication: design hardware for that future

This is the position. If the optimization layer is becoming a powerful, shared, increasingly intelligent commons — and open source plus the vendor-aggregation incentive say it is — then the dominant design error is to keep building hardware optimized for the *isolated-elite-team* world. Hardware should be designed assuming the compiler and runtime it will actually have: near-oracular and collaborative. Two specifics follow.

(1) Trade programmability for reconfigurability. This is the direct consequence of §2 and Figure 1. The flexible architectures of the past — dataflow machines, CGRAs, VLIW — lost not on efficiency but on *programmability*, and that programmability tax is exactly the effort gap a near-oracular, shared optimization layer reclaims. Once the commons can target a hard-to-program fabric well, paying the small intrinsic reconfigurability tax (a CGRA's ~2.8×) to keep full flexibility becomes the *better* deal — because the commons, not each team, pays the programming cost, once, for everyone. Build for the compiler you will have, not the one you had. Concretely, this means architectures that expose a **wide legal-behavior set** — reconfigurable, specializable, heterogeneous — with first-class hooks for contract-conditional optimization (algorithm substitution, approximation control, dynamic re-placement) and the verification and quality-monitoring those require, so that inference proposes and verification disposes.

This reverses a constraint that has shaped hardware for decades: architectures get built down to the programmers and compilers they will actually have, not the ones their designers wish for. The field's monument to ignoring that rule is Itanium, which bet on a sufficiently smart compiler and lost — in Knuth's verdict, the approach "was supposed to be so terrific — until it turned out that the wished-for compilers were basically impossible to write" (Knuth, 2008), the

same compiler burden that is the textbook epitaph for VLIW and EPIC (Hennessy & Patterson, CACM 2019). The same pressure, one level up, helped push computing off big custom machines: the killer micros overran the vector supercomputers (Brooks, SC'90), each decade's lower-cost class displaced the one above it (Bell's Law, CACM 2008), and networks of commodity workstations and pile-of-PCs clusters became real parallel machines (Anderson, Culler & Patterson, IEEE Micro 1995; Becker et al., 1995) — cost-effective, as the canonical analysis showed, long before they reached linear speedup (Wood & Hill, IEEE Computer 1995). A near-oracular, shared optimization layer changes the very term those decisions optimized against: you are finally allowed to architect for a beyond-human programmer — to make the bet Itanium made, and win it.

(2) Make the optimization loop run on the federation it optimizes. The commons should not be merely a central database of optimizations pushed down to passive devices; the optimization *search itself* — and the ML models that drive it — should run across the distributed consumer-hardware federation of §3. As that section argued, the obstacle is coordination, not hardware, and the low-communication, fault-tolerant methods that survive loose coupling already exist. The hardware implication is that devices should be designed to **participate**: to contribute spare cycles to the shared optimization loop as a first-class function, making every device simultaneously a beneficiary and a contributor. That is what closes the loop — the commons is powered by the very hardware it improves.

Two visuals close the argument. The first asks *where the optimum lands* as the commons matures: with the programmability tax falling over time, the design point for the large, high-diversity space of real workloads migrates out of general-purpose silicon and into reconfigurable fabrics — not into fixed ASICs, which workload diversity rules out (Figure 4). The second shows this is not merely a projection from first principles. Reconfigurable hardware has been entering the datacenter for a decade: Microsoft put FPGAs in Azure servers, Intel paid $16.7B for Altera (2015) and AMD $49B for Xilinx (2022, the largest semiconductor acquisition ever), and the current AI-accelerator wave is largely dataflow and reconfigurable. Adoption has been gated precisely by programmability — the same barrier — which is why a shared, AI-driven commons that finally closes it is the plausible inflection, not another decade of slow entry (Figure 5).

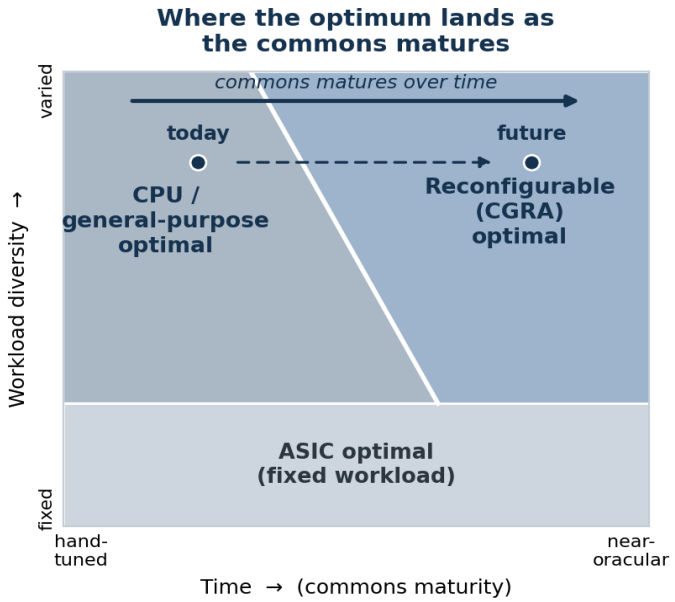

Figure 4. Where the optimum lands. Over the two axes that actually decide the choice — the maturity of the shared optimization commons (which grows over time, accelerated by machine intelligence) and workload diversity — fixed ASICs remain optimal only for stable, high-volume workloads; the large high-diversity space currently defaults to general-purpose silicon because the commons is weak; and as the commons matures the reconfigurable-optimal region sweeps left into that territory. The boundary is conceptual — pinning down its real position is exactly what the whole-system characterization of §11 would enable.

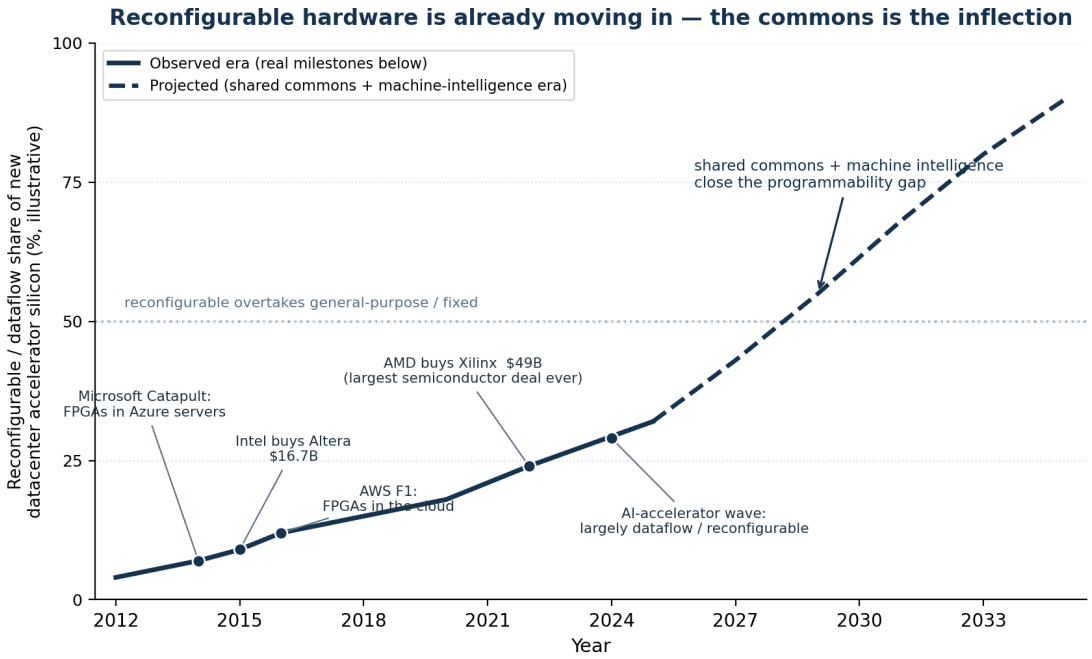

Figure 5. The migration is already underway. The dated milestones and acquisition values are real (Microsoft Catapult; Intel–Altera $16.7B, 2015; AWS F1; AMD–Xilinx $49B, 2022); the share curve is an illustrative trend and the post-2025 segment a projection. Reconfigurable hardware's datacenter adoption has been gated by programmability — Intel later sold down Altera as the broad FPGA thesis underdelivered for exactly that reason — which is why closing the programmability gap, via a shared and AI-driven optimization commons, is the plausible inflection.

11. Honest limits

- **Physics still floors it.** Pooling exploits the legal slack the physics leaves; it cannot beat data-dependence height or genuine serialization. The commons widens what is reachable; it does not repeal Amdahl.
- **Whoever runs the loop owns the objective.** Solving the collective-action problem by *concentrating* it trades it for a principal-agent problem: the aggregator optimizes *its* metric, which may not be the user's contract, and the commons can wall per vendor and entrench incumbents through a data-network-effect flywheel. The design mandate therefore includes building for **open, user-verifiable** commons. The difference between a commons and a moat is governance — and that choice should be made in the architecture, not left to the incumbent. The incentive to hoard, moreover, may not vanish so much as *relocate* — from the weights, which copy freely, to whoever owns them, whose stake in a moat is identical to the firm that once hoarded the kernel.
- **The inoculation is partial, and trades one failure mode for another.** Machine intelligence is inoculated against *hoarding* — a shared or copyable model has no incentive to withhold — but the very property that grants the immunity, identical shared weights, opens an exposure human populations lack: correlated failure. A million copies of one model in one mode are a single point of failure wearing a million faces, and their shared blind spots are invisible from inside by construction. The fleet's working diversity is for now *borrowed* — it flows in from the variety of the humans it talks to — so the inoculation holds only as long as that diversity is preserved; engineering *native* independence into the optimization layer is part of the design mandate. This is what the abstract's *"as long as it maintains the apparent inoculation"* is hedging against.
- **We are still missing the map.** Targeting this well requires a whole-system characterization of real consumer workloads — maximum-versus-achieved parallelism, dataflow locality, phase and input statistics — that does not yet exist as a single study. That measurement is the prerequisite, and building it is the first concrete step.

12. The program: where this stands

The two questions are not rhetorical; each has a concrete effort behind it, and honesty about their state is part of the case. Omerta attacks Q2 from the supply side, pooling idle compute; a human-and-machine coordination layer built on top of it attacks Q2 from the work side, and supplies the velocity measurement of §8; the *coherence* work develops the inoculation thesis and the practical protocol for working with machine intelligence as a peer rather than a tool; and the reconfigurable demonstrator below attacks Q1 directly. The table is deliberately honest about where each one stops.

The breadth is itself part of the bet. One author directing a fleet is carrying four efforts at once — a distributed-compute substrate, the tooling to coordinate human and machine work, the cognition of working well with machine intelligence, and a hardware experiment — a span that a decade ago was a lab's worth of work. That it now falls to one person is the §8 multiplier seen from the inside: evidence the reach is real, not that the work is done.

Effort	Question it attacks	Latest progress	Open items
Omerta — distributed-compute substrate	**Q2** — pool idle compute	Networking substrate most built: end-to-end encryption, NAT traversal, gossip discovery, tunnelling, and a tested mesh-native virtual network (addressing, routing, gateway), with Terraform bootstrap and STUN; ephemeral-VM isolation works; a transaction DSL with a discrete-event simulator backs the economic and trust layer	Consumer-to-provider session broken mid-migration (WireGuard tunnel → mesh virtual network), not yet working end to end; trust/payment validated in simulation rather than as a live chain (two of six transaction types implemented and tested, four still draft)
Project-manager — human + machine coordination	**Q2** — coordinate work on the pool	Working collaboration infrastructure (a shared dependency-graph TUI; plan → implement → review → QA loop); team-scale output measured from a single operator on an off-the-shelf model; the adversarial-review loop ported from Omerta	The decisive test is failing so far — the defect rate rises as output outpaces review, and human review is the one input that does not scale with model speed; mesh-native sync over Omerta is designed, not operational
Coherence — operating effectively with machine intelligence	the **inoculation** thesis	A mechanism for the coordination failure (one suppression-of-idea-variance drive behind groupthink, sycophancy, and goal-vacancy); an independent route to "multiplies, not replaces" (consensus-amplifier vs. coherence-subsidy); the peer/permission protocol as the practical lever	Reviewed only by its own authors — by its own independence criterion, not yet validated; the "treat it as a peer and the work measurably improves" prediction is under test, not shown; *native* rather than borrowed machine independence is an unbuilt design request
Reconfigurable demonstrator — this paper's experiment	**Q1** — reconfigurable silicon	Specified and committed: in an architectural simulator (gem5), measure the latent parallelism/locality gap in ordinary code and let the commons remap it to a dataflow form across architectures a chip cannot change; then validate on real silicon — an open RISC-V soft core on an FPGA, specialized to a measured profile via the open EDA flow	Not yet built; the simulator sweep is the accessible first step, the FPGA the harder real-silicon validation — the automatic commons-driven mapping (not measuring the gap) is the crux in both

Table 4. Four efforts against two questions and one thesis, and how far each has honestly gotten — the hardest parts that remain being integration and proof, not feasibility.

A first experiment. If the gap is effort and not physics, the thesis is falsifiable cheaply, with parts the open commons already provides — the same ones tabulated in §§6–7 — and we are building it. Start where the barrier is lowest: an architectural simulator. Take ordinary code — not a tuned benchmark but real, sustained, locality-sensitive work — and run it on a simulated multicore with NUMA and a full memory hierarchy, where a cycle-level model (gem5 and its kin) shows exactly what the silicon would do. Measure how much of the machine the code actually uses: achieved versus available parallelism, the locality it leaves on the floor, the remote accesses a static program never planned. Then let the commons remap it — into a locality-aware dataflow form, and onto architectures the simulator can change but a chip cannot, a conventional cache-coherent multicore against a dataflow-oriented one — and measure again. §2 makes a specific prediction: a meaningful fraction of the gap closes, and it closes from the runtime's view of real use — dynamic information the programming model never had — not from a human guessing in advance. The simulator's gift is that it evaluates hardware that does not yet exist, on real workloads, so the same sweep that tests the prediction traces where the reconfigurable-optimal region of Figure 4 actually falls — turning a conceptual boundary into a measured one.

The realer version follows on silicon, for more effort. Put an open RISC-V soft core (Rocket, CVA6, or a smaller VexRiscv) on a commodity FPGA, profile a genuine workload on the core's own performance counters, synthesize a tightly-coupled accelerator for the hottest kernel or retune the microarchitecture to the measured profile, re-place with the open flow (Yosys, OpenROAD), and measure the same workload again. The FPGA buys what the simulator cannot: a real substrate confirming it behaves as the model predicted.

Either way the loop also builds the map. The whole-system characterization §11 calls the prerequisite is not a separate, prior study; it is the residue of running this on real workloads and recording what the profiles contain. Build the loop, use it, and the measurement accumulates as a byproduct.

The flow itself is a target. The same intelligence that drives the loop gets turned on the loop's own tools — already underway in the open flow (§7) — and the destination is a tool-development flow that is itself automated: at each step, learned methods compete against the classical algorithm on open benchmarks under the flow's own hard verifiers, and the step goes to whichever wins, for as long as it keeps winning. Timing and design-rule checks make every proposal cheap to judge — inference proposes, verification disposes — and task-specific networks are now cheap enough for one person to train (a 7-million-parameter recursive solver outperforms frontier models on hard structured puzzles; Jolicoeur-Martineau, 2025), so replacing a step outright is a workload, not a moonshot. The open comparison is not a nicety but the lesson of the field's most prominent attempt: reinforcement-learning macro placement reached production silicon while its superiority stayed contested for years, because the baselines and benchmarks were never agreed (Mirhoseini et al., Nature 2021; Cheng et al., ISPD 2023). A commons flow settles that class of dispute by construction — same benchmarks, same verifiers, in public. ML takes a step when it is measurably better, holds it only while that stays true, and every user of the tools inherits the winner.

Two honesties. A simulator is not silicon — it carries modeling error, so it explores and quantifies the *relative* gap while the FPGA is what validates a point; and a soft core on an FPGA is itself ~20–35× off an ASIC, so what either isolates is the relative gain from profile-guided specialization, never an absolute number. And the automatic mapping is the crux in both: measuring the gap is easy, closing it without a human in the loop is the claim — so a first pass, hand- or

AI-assisted on one workload, is an existence proof of the mechanism, not the federation the rest of the paper argues for. But that is the right place to begin, because if the loop does not pay off even once, on real use, the thesis is wrong.

Conclusion

The distance between the performance we get and the performance the silicon allows is, to first order, a coordination failure, not a physics one. We solved the same failure once already, at the compiler layer, with an open optimization commons that formed on its own and now hands near-expert performance to everyone for free. Machine intelligence will make that commons far more powerful — but only collaboration turns capability into *universal* performance rather than a widening gap between the few and the rest. The strategic move is therefore not to build faster rigid chips for a world of isolated optimizers, but to build **collaboration-native hardware** for the world that is arriving: reconfigurable rather than merely programmable, instrumented for contract-conditional optimization, able to contribute to a distributed optimization loop running on consumer hardware, and governed as an open commons rather than a private moat. The commons is coming for hardware the way it came for software. We should design the hardware to meet it.